

Universitat d'Andorra

REPTE M5 ANY 2023/2024

Disseny i programació d'aplicacions informàtiques

Preparat per

Eric Torrontera Ruiz

etorrontera@uda.ad

12 de gener de 2023

TAULA DE CONTINGUTS

1. Breu presentació del Framework utilitzat	3
2. Descripció del problema a modelar	4
3. Diagrama de classes	6
4. Sistema de fidelització amb punts i descomptes (<i>Patró State</i>)	8
5. Sistema de creació llibres pel testeig de la API (<i>Factory + Template</i>)	12
6. Patrons incorporats a Laravel	14

Framework utilitzat: Laravel Patrons incorporats a Laravel

Per tal de realitzar el repte utilitzarem Laravel. És un marc de treball PHP creat per Taylor Otwell que proveeix una base de codi ja construïda, facilita el desenvolupament i proporciona una estructura ja creada amb la qual treballar. En els últims anys ha guanyat popularitat gracies a la seva elegància, sintaxi expressiva i conjunt complet d'eines.

Comentar que segueix el patró de disseny MVC (Model-Vista-Controlador) i ofereix característiques com **Eloquent ORM**, Blade templating engine, un sistema d'enrutament clar i un conjunt d'eines de desenvolupament anomenades Artisan. Laravel promou la productivitat i la facilitat d'ús, convertint-lo en una opció popular per al desenvolupament web modern.

Descripció del problema a modelar

Per a començar a modelar el sistema de venda de llibres amb Laravel, primer necessitem entendre els requeriments del problema de manera clara i precisa.

L'empresa escollida és una petita llibreria que vol entrar al món del ecommerce i obrir un portal de venda de llibres per internet. Actualment només es dedica a la venda de llibres. Es desitja desenvolupar una plataforma de venda de llibres en línia utilitzant Laravel. La plataforma permetrà als usuaris comprar llibres amb descomptes especials, els quals varien segons el tipus d'usuari.

Usuaris: Hi ha quatre tipus d'usuaris: normals, originals, genuïns i llegenda. Cada tipus d'usuari té un descompte addicional específic sobre el preu dels llibres. El usuari podran canviar de categoria en funció del que han gastat al web.

- **Normals:** Tenen un descompte del 0%. [0€ , 50€)
- **Originals:** Tenen un descompte del 5%. [50€ , 100€)
- **Genuïns:** Tenen un descompte del 10%. [100€ , 150€)
- **Llegenda:** Tenen un descompte del 15%. [150€ , ∞€)

Llibres: Cada llibre té un preu base. Els descomptes s'apliquen de manera addicional al preu base del llibre segons el tipus d'usuari. Els descomptes són dinàmics i poden canviar en el futur. Existeixen dos tipus de llibres: Llibres normals i en liquidació. Aquets últims que tenen un 50% de descompte sobre el preu base.

Cada llibre té associat un preu unitari. El client podrà comprar més d'un exemplar del mateix llibre, fins un màxim determinat pel seu estoc.

Quan un client realitza una comanda en una determinada data, pot comprar tants productes com vulgui però amb un màxim determinat pel seu estoc. Quan una comanda es formalitza s'ha de guardar el registre. L'enviament serà gestionat per altres empreses de transport com "Correus espanyols" o "La Poste".

Requeriments:

Usuaris: S'ha d'implementar un sistema d'autenticació perquè els usuaris puguin iniciar sessió i registrar-se. Cada usuari ha de tenir un perfil que indiqui el seu tipus (normal, original, genuí, llegenda).

Llibres: Ha d'existir una base de dades de llibres amb informació com a títol, autor, preu basi, etc.

Descomptes: Ha d'haver-hi un mecanisme per a aplicar descomptes de manera dinàmica. Els descomptes han de ser gestionats pel sistema i podrien canviar en el futur.

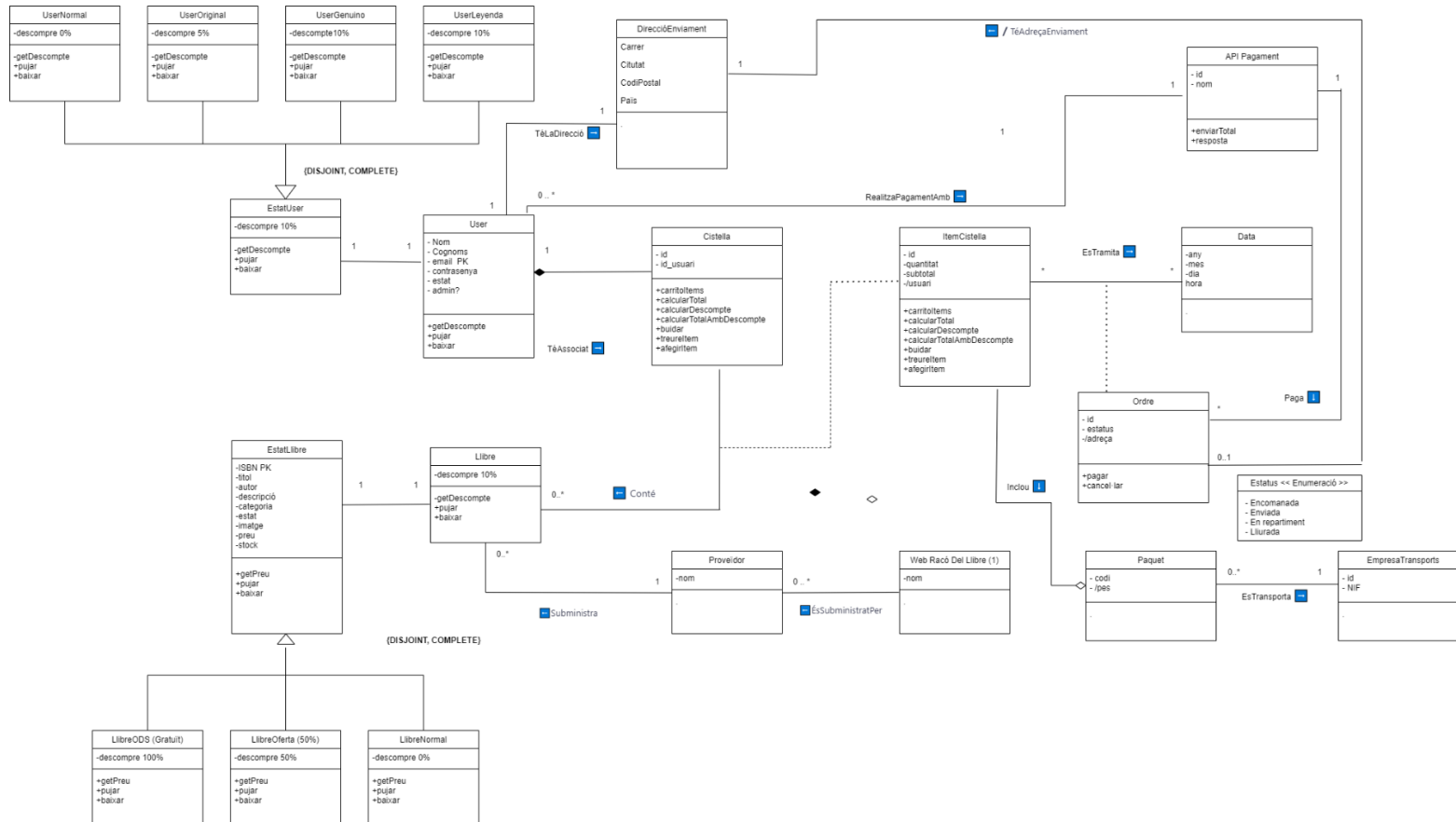
Compra de Llibres: Els usuaris han de poder agregar llibres a la cistella de compres. El sistema ha de calcular automàticament el descompte aplicable segons el tipus d'usuari. S'ha de mostrar el preu final abans de la confirmació de compra.

Comandes: Implementar un sistema per a gestionar i seguir l'estat de les comandes.

Restriccions Textuals:

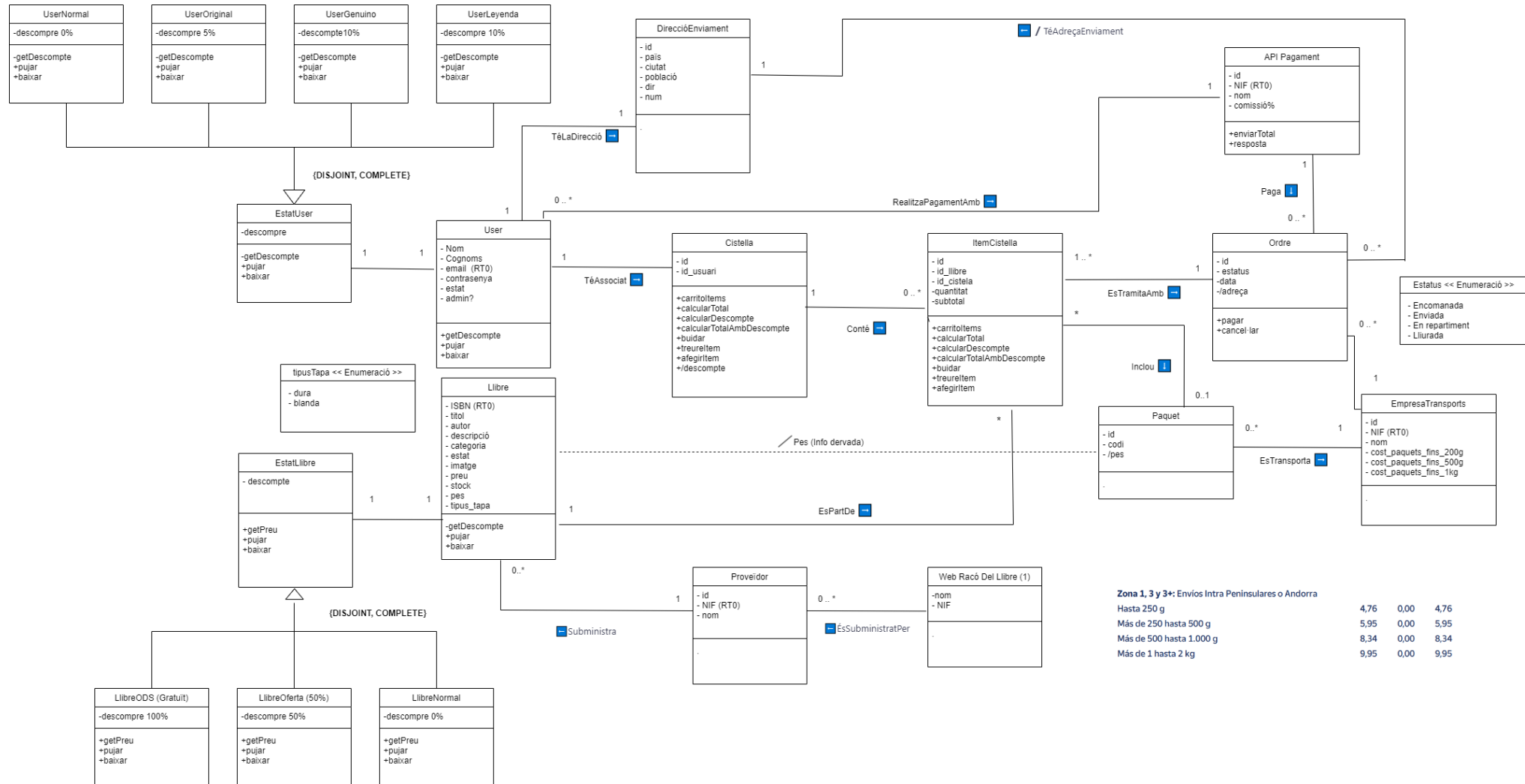
- **RT0 (clau externa):** (Client, email), (Llibre, ISBN)
- **RT1 (enunciat):** Els descomptes han d'aplicar-se de manera automàtica en agregar llibres a la cistella.
- **RT2(enunciat):** Un usuari pot tindre múltiples llibres en la seva cistella de compres.
- **RT3(enunciat):** El màxim que pot ser afegit a la cistella és l'estoc disponible del producte.

Diagrama de classes d'especificació



Creem les classes associatives quan les relacions entre classes tenen atribut i mètodes propis. Això significa que la relació en sí mateixa porta informació addicional que no pot ser representada completament per les classes involucrades. Agregació: relació "tot-part" més feble, les parts poden existir independentment. Composició: relació més forta, les parts no poden existir sense el tot.

Diagrama de classes de disseny



Enllaç als diagrames per si no es poden veure bé: https://app.diagrams.net/#G1o_MCU6CHjv04EjDa4g4E2ldPC2ASTbFI

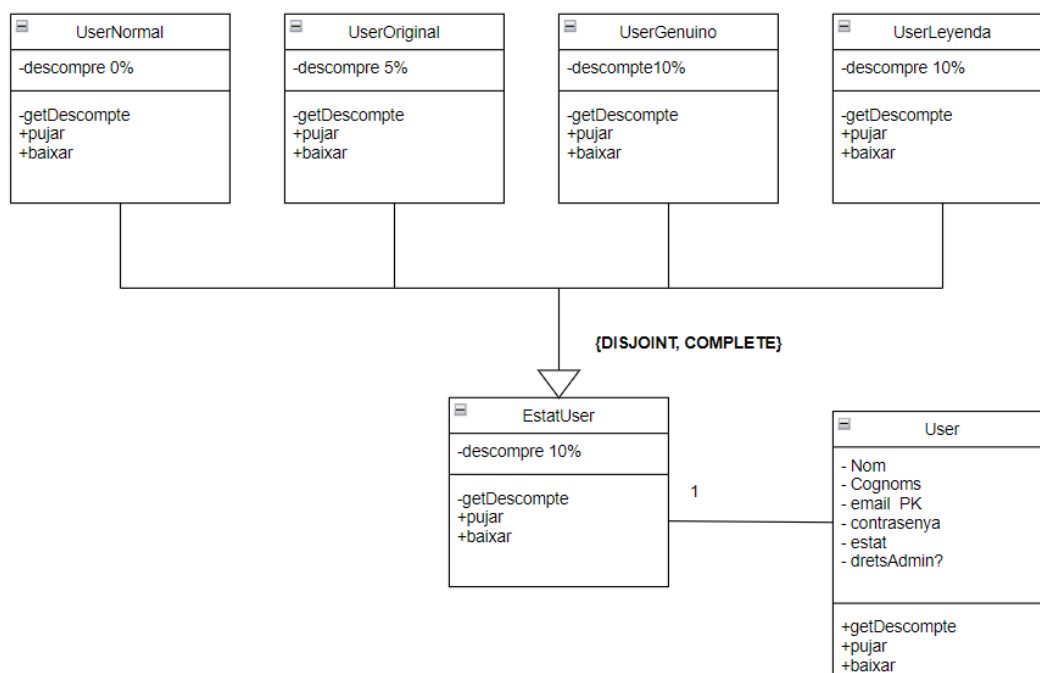
Sistema de fidelització amb punts i descomptes

En aquest apartat es mostrarà una de les funcionalitats del sistema. Concretament ens centrarem en el sistema de punts i descomptes.

El sistema d'usuaris està dissenyat amb la finalitat de proporcionar una experiència personalitzada als clients del nostre lloc web de venda de llibres. Hi ha quatre categories distintes d'usuaris, cada una amb un nivell específic de descompte en funció de les seves despeses acumulades. Aquestes categories inclouen:

- **Normals:** Tenen un descompte del 0%. [0€ , 50€)
- **Originals:** Tenen un descompte del 5%. [50€ , 100€)
- **Genuïns:** Tenen un descompte del 10%. [100€ , 150€)
- **Llegenda:** Tenen un descompte del 15%. [150€ , ∞€)

Per gestionar aquest sistema de categories i descomptes, s'ha optat per utilitzar el **patró de disseny State**. Això permet una transició suau entre els diferents estats dels usuaris basant-se en les seves activitats de compra. Cada estat representa una categoria específica i defineix el descompte que s'aplica segons les despeses acumulades. Així, els usuaris poden canviar dinàmicament de categoria a mesura que fan més compres, brindant-los incentius i recompenses que s'ajusten al seu comportament i fidelitat al lloc web. Aquest enfocament proporciona una gestió eficaç dels estats dels usuaris i millora la seva experiència de compra alhora que promou la retenció de clients.



Laravel porta incorporat Eloquent ORM, una característica que simplifica la interacció amb bases de dades. Ens permet accedir als registres com si fossin objectes a través dels models Eloquent que creem. Eloquent ens permet definir relacions entre models. Aquestes relacions poden ser un a un, un a molts o molts a molts. Dit això per tal de poder traslladar el estat de la base de dades a las instàncies en si mateixes hem fet ús d'Eloquent i els models de Laravel. La següent funció *getEstatUsuari()* es trucada al instanciar un Usuari.

```
43     public function getDescompte()  
44     {  
45         $estatUsuari = $this->getEstatUsuari();  
46         return $estatUsuari->getDescompte();  
47     }  
48  
49     protected function getEstatUsuari(): EstatUser {  
50  
51         switch ($this->estatus)  
52         {  
53             case 'Normal':  
54                 return new NormalUser();  
55             case 'Original':  
56                 return new OriginalUser();  
57             case 'Genuino':  
58                 return new GenuinoUser();  
59             case 'Leyenda':  
60                 return new LeyendaUser();  
61             default:  
62                 return new NormalUser();  
63         }  
64     }  
65 }
```

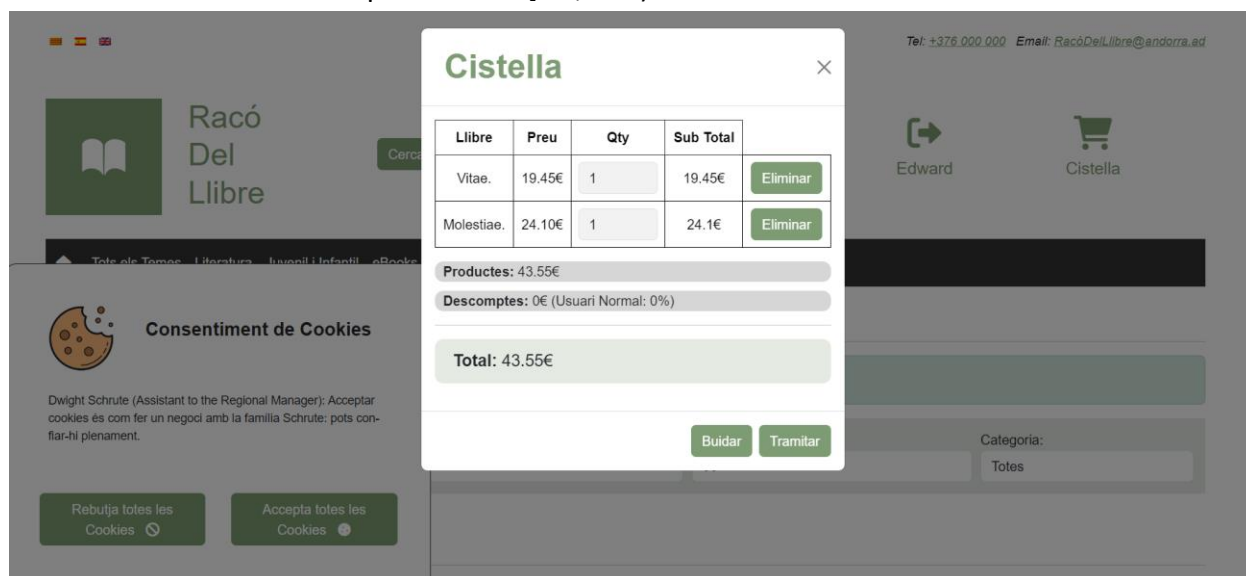
Il·lustració 1 – Funcions del model Usuari de Laravel

El mètode abstracte *getDescompte()* de la classe *EstatUsuari* variarà en funció de la instància assignada i per tant variarà el seu percentatge de descompte. Aquest descompte serà accés per la Cistella per tal de plasmar el descompte:

```
$descompteUsuari = $this->user->getDescompte();
```

Aquí podem veure una imatge de la implementació al web dels descomptes sobre la cistella:

Normals: Tenen un descompte del 0%. [0€ , 50€)



Originals: Tenen un descompte del 5%. [50€ , 100€)

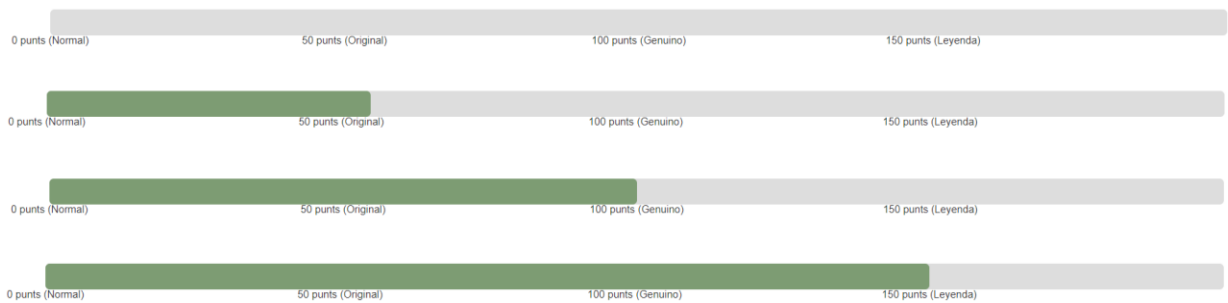
Genuïns: Tenen un descompte del 10%. [100€ , 150€)

Llegenda: Tenen un descompte del 15%. [150€ , ∞€)



Com es evident al tramitar la ordre també s'ha establert la navegabilitat per tal de poder tramitar l'ordre amb aquest descompte. A la dreta podem veure l'aplicació de la **RT3(enunciat)**: El màxim que pot ser afegit a la cistella és l'estoc disponible del producte.

L'usuari per cada comanda pagada acumularà 1 punt per euro gastat i podrà veure una barra de progrés al seu dashboard:



Així, els usuaris poden canviar dinàmicament de categoria a mesura que fan més compres, brindant-los incentius i recompenses que s'ajusten al seu comportament i fidelitat al lloc web. Aquest enfocament proporciona una gestió eficaç dels estats dels usuaris i millora la seva experiència de compra alhora que promou la retenció de clients.

Hola Montse!

Usuari: Leyenda (15% de descompte addicional)

Punts: 198

(Normal)

50 punts (Original)

100 punts (Genuino)

150 punts (Leyenda)

Veuve panel d'usuari

Log out

Panel de Montse

Categoria: Leyenda

Tens un descompte addicional del 15% sobre les teves compres! Aprofita-ho al màxim 🤩

Punts acumulats: 198

(Normal)

50 punts (Original)

100 punts (Genuino)

150 punts (Leyenda)

Cistella

La teva cistella està buida 🛒

Historial d'ordres

Data: 15/01/2024 21:03:51 Total amb descomptes: 123.25€ Order No.1

Llibre	Preu unitari	Qty	Subtotal
Quia commodi.	25.32€	2	50.64€
Quia commodi.	25.32€	1	25.32€

Sistema de creació llibres pel testeig de la API (Factory + Template)

Per tal de poder testejar de manera correcta les funcionalitats del sistema, com poden ser els filtres a la pàgina de productes, s'ha fet ús dels patrons que te incorporat Laravel. Hem creat 1000 registres en qüestió de segons amb la combinació del Patró Template i Factory Method.

En el codi del factory “*LlibreFactory*”, s'implementa el patró Template al definir un esquema bàsic (*definition*) per a la creació d'instàncies de la classe Llibre. Aquest esquema actua com una "plantilla" que estableix l'estructura i les característiques dels objectes Llibre que es generaran durant les proves de la base de dades.

```
return [  
    'titol' => $this->faker->text(15),  
    'descripcio'=> $this->faker->paragraph(),  
    'categoria'=> $this->faker->randomElement(['Juvenil', 'Còmics i manga', 'Infantil', 'Novel·la', 'Poesia', 'Teatre', 'Ciència-ficció',  
    'subcategoria'=> $this->faker->randomElement(['Novetats', 'Ofertes', 'Best Sellers']),  
    'autor' => $this->faker->name(),  
    'preu' => $this->faker->randomFloat(2, 5, 30),  
    'isbn' => $this->faker->isbn13(),  
    'imatge' => $imagenUrl,  
    'stock' => $this->faker->numberBetween(1, 100)  
    // 'imatge' => 'https://picsum.photos/300/400?random=' . $this->faker->unique()->numberBetween(1, 1000),  
];
```

Seguidament hem fet ús del Factory Method a Laravel i Seeders: En el context de Laravel i els factories, s'utilitza implícitament el patró Factory Method. El mètode definition en el factory actua com un factory method al ser responsable de la creació d'objectes Llibre. Laravel proporciona una estructura on aquest mètode s'invoca per crear instàncies de models, en aquest cas el model Llibre.

```
1  <?php  
2  
3  namespace Database\Seeders;  
4  
5  //use Illuminate\Database\Console\Seeds\WithoutModelEvents;  
6  use App\Models\Llibre;  
7  use Illuminate\Database\Seeder;  
8  
9  class LlibreSeeder extends Seeder  
10 {  
11     /**  
12      * Run the database seeds.  
13     */  
14     public function run(): void  
15     {  
16  
17         Llibre::factory(1000)->create();  
18     }  
19 }  
20
```

Ara amb la comanda `php artisan migrate:fresh --seed` crearem 1000 registres. Yey!

Com podem observar hem aconseguit crear 1000 registres que podem instanciar.

The screenshot displays a database management interface for a table named 'llibres'. The top status bar indicates 'Showing rows 0 - 24 (1000 total, Query took 0.0005 seconds)'. Below this, the SQL query 'SELECT * FROM `llibres`' is shown. The interface includes controls for 'Number of rows' (set to 25), a 'Filter rows' search bar, and a 'Sort by key' dropdown (set to None). The table columns are: id, isbn, titol, autor, descripcio, categoria, subcategoria, stock, imatge, and preu. The first three rows of data are visible. A modal window is open over the table, showing details for a book with ISBN 9791336161794, author Jaquan Bashirian MD, category Idiomes, and stock 74. The modal also displays a book cover image and a price of 14.85€.

	id	isbn	titol	autor	descripcio	categoria	subcategoria	stock	imatge	preu
☐	1	9780421311503	Sed et aut.	Pietro Cummerata V	Unde sint cumque fugit aut est earum delogiti	Ciència-ficció	Ofertes	14	img/llibre0.png	6.74
☐	2	9780326388501	Quisquam.							
☐	3	9793141242811	Et excepturi.							

Sapiente.

ISBN: PLANETA-9791336161794
Autor: Jaquan Bashirian MD
Categoria: Idiomes
Estoc: 74

Quas unde ipsa nemo quisquam ut eum excepturi quo. Non occaecati molestiae fugit cumque dolor qui. Pariatur odio at quae eos officiis sequi.

Preu: 14.85€

[Alegir a la cistella](#) [Tancar](#)

En resum, l'ús del patró Template garanteix coherència en l'estructura de les dades de prova, mentre que el factory method abstraïx la creació d'instàncies, permetent l'adaptabilitat i l'extensibilitat del codi. Ambdós conceptes contribueixen a un disseny robust i mantenible en el desenvolupament de proves de base de dades a Laravel.

Patrons incorporats a Laravel

A part dels patrons anteriorment comentats Laravel incorpora altres patrons de disseny i característiques de programació que faciliten el desenvolupament. Algunes d'aquestes característiques i patrons inclouen:

- **Patró de Repositori (Repository Pattern):** Encoratja l'ús de repositoris per gestionar l'accés a les dades. Aquest patró permet centralitzar la lògica d'accés a la base de dades en un lloc específic, facilitant les proves i el manteniment.
- **Patró de Filtre (Filter Pattern):** Laravel fa ús dels filtres (middlewares) per processar sol·licituds abans d'arribar al controlador. Aquest patró permet aplicar diferents capes de processament a les sol·licituds HTTP.
- **Patró Observador (Observer Pattern):** Eloquent, el sistema ORM de Laravel, utilitza l'Observador per permetre que les classes escoltin esdeveniments del model i reaccionin en conseqüència. Això facilita la implementació d'accions posteriors a l'emmagatzematge o actualització dels models.
- **Patró Singleton:** Laravel utilitza el mític patró singleton en alguns dels seus components, garantint que només existeixi una única instància d'alguns serveis clau.
- **Patró de Migració de Base de Dades:** Laravel fa servir un enfocament de migració de base de dades per gestionar els canvis d'esquema. Això ajuda a mantenir la consistència de la base de dades a llarg termini.
- **Model-Vista-Controlador (MVC):** Com hem comentat al principi del document, Laravel segueix l'arquitectura MVC. Aquesta separació en capes permet gestionar la lògica del negoci (controlador), la presentació (vista) i la interacció amb la base de dades (model) de manera independent.
- **Active Record (Eloquent):** Eloquent és l'ORM (Mapeig Objecte-Relacional) de Laravel i segueix el patró Active Record. Això significa que els models Eloquent són utilitzats per interactuar amb les taules de la base de dades i proporcionen una interfície orientada a objectes per gestionar aquesta interacció.